

PL/SQL Training

SESSION 7: INTRODUCTION TO BULK COLLECT

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

SESSION 7

INTRODUCTION TO BULK COLLECT



- The Row-by-Row Problem
- Possible Solutions
- Cursor FOR Loops and BULK COLLECT
- RETURNING and BULK COLLECT
- Introduction to FORALL
- SQL%ROWCOUNT and SQL%BULK_ROWCOUNT
- Errors and FORALL
- Continuing Past Errors with SAVE EXCEPTIONS
- SQL%BULK_EXCEPTIONS
- Table Function& BULK

Context switches and performance

In Oracle PL/SQL, a context switch is a performance-critical concept that occurs when control passes between the SQL engine and the PL/SQL engine.

Excessive context switching can cause serious performance degradation, especially in data-heavy applications.

What Is a Context Switch?

Oracle runs PL/SQL and SQL using two separate engines:

- PL/SQL engine – Executes procedural logic (loops, conditionals, variables, etc.)
- SQL engine – Executes SQL statements (SELECT, INSERT, UPDATE, DELETE)

A context switch happens when:

- PL/SQL invokes a SQL statement (even SELECT ... INTO)
- SQL returns control and results to PL/SQL



Each switch has overhead, and if done inside loops, it creates performance bottlenecks.

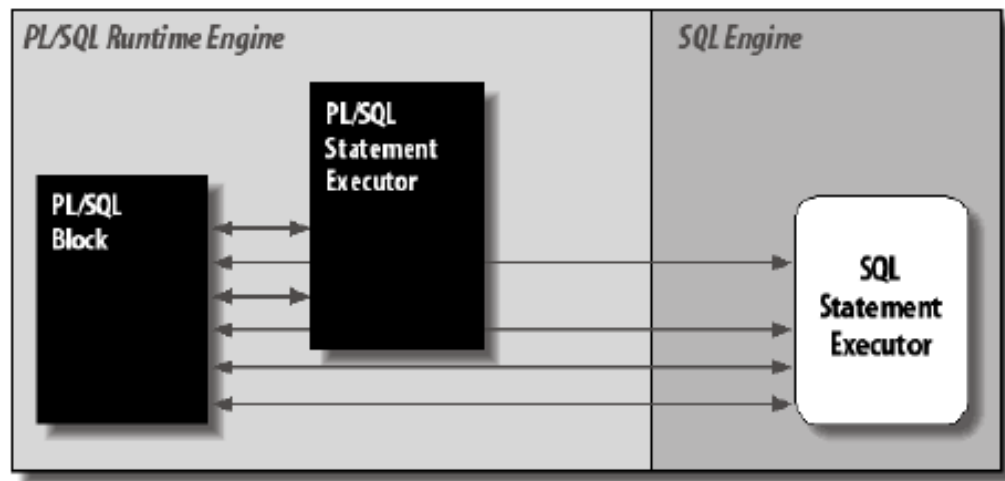
Example of Expensive Context Switching

```
FOR rec IN (SELECT emp_id FROM employees)
LOOP
  -- Executes 1 UPDATE per row = 1 context switch each time
  UPDATE employees
  SET salary = salary * 1.1
  WHERE emp_id = rec.emp_id;
END LOOP;
```

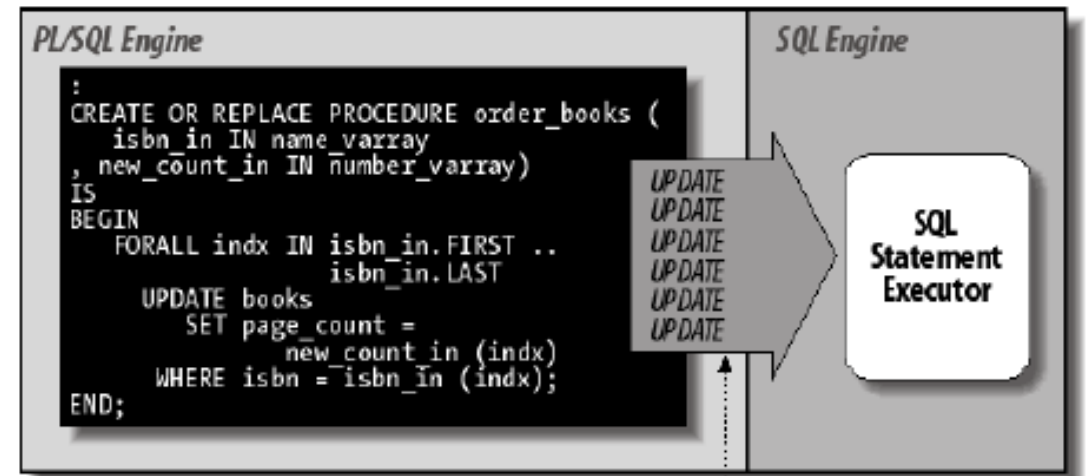
- ✓ This works, but Oracle switches between SQL and PL/SQL engines for every row, causing slow-by-slow performance.

Bulk Data Processing

Oracle introduced a significant enhancement to PL/SQL's SQL-related capabilities with the **FORALL** statement and **BULK COLLECT** clause for queries. Together, these are referred to as *bulk processing* statements for PL/SQL.



This transfer of control between the PL/SQL and SQL engines is called a *context switch*. Each time a switch occurs, there is additional overhead.





Eliminating Context Switches: Use BULK Techniques

- ✓ **BULK COLLECT (reduce SQL → PL/SQL switches) – Read Many Rows at Once**
- ✓ **FORALL (reduce PL/SQL → SQL switches) - Perform DML on Many Rows at Once**

High Speed Querying with BULK COLLECT

... **BULK COLLECT INTO** *collection_name*[, *collection_name*] ...

- You can use **BULK COLLECT** keywords in any of the following clauses: **SELECT INTO**, **FETCH INTO**, and **RETURNING INTO**.
- The SQL engine automatically initializes and extends the collections you reference in the **BULK COLLECT** clause. It starts filling the collections at index 1, inserts elements consecutively (densely), and overwrites the values of any elements that were previously defined.
- You can't use the **SELECT...BULK COLLECT** statement in a **FORALL** statement.
- **SELECT...BULK COLLECT** will not raise **NO_DATA_FOUND** if no rows are found. If the query returns no rows, the collection's COUNT method will return 0.

Bulk Data Processing



Using the RETURNING clause with bulk operations

The **RETURNING** clause allows you to obtain information from a DML statement. **RETURNING** can help you avoid additional queries to the database to determine the results of DML operations that just completed.

High Speed DML with FORALL

FORALL *index* **IN**

[*lower_bound ... upper_bound* |

INDICES OF *indexing_collection* |

VALUES OF *indexing_collection*]

Bulk Data Processing



- The body of the **FORALL** statement must be a single DML statement—an **INSERT**, **UPDATE**, **DELETE**, or **MERGE**
- The DML statement must reference collection elements, indexed by the *index* variable in the **FORALL** statement. The scope of the *index* variable is the **FORALL** statement only; you may not reference it outside of that statement.
- Do not declare a variable for *index*. It is declared implicitly as **PLS_INTEGER** by the PL/SQL engine.
- The lower and upper bounds must specify a valid range of consecutive index numbers for the collection(s) referenced in the SQL statement. Sparsely filled collections will raise the following error:

ORA-22160: element at index [3] does not exist

Name	Description
SQL%FOUND	Returns TRUE if the last execution of the SQL statement modified one or more rows.
SQL%NOTFOUND	Returns TRUE if the DML statement failed to change any rows.
SQL%ROWCOUNT	Returns the total number of rows processed by all executions of the SQL statement, not just the last statement.
SQL%ISOPEN	Always returns FALSE and should not be used.
SQL%BULK_ROWCOUNT	Returns a pseudo-collection that tells you the number of rows processed by each corresponding SQL statement executed via FORALL.
SQL%BULK_EXCEPTIONS	Returns a pseudo-collection that provides information about each exception raised in a FORALL statement that includes the SAVE EXCEPTIONS clause.



Benchmark Example (Time Test)

✗ Row-by-row (with context switching):

✗ Row-by-row (with context switching):	✓ Bulk version (minimized switches):
<pre>FOR rec IN (SELECT emp_id FROM employees) LOOP DELETE FROM employees WHERE emp_id = rec.emp_id; END LOOP;</pre>	<pre>SELECT emp_id BULK COLLECT INTO v_ids FROM employees; FORALL i IN v_ids.FIRST .. v_ids.LAST DELETE FROM employees WHERE emp_id = v_ids(i);</pre>



Graphical Summary (Conceptual)

Approach	Context Switches	Performance
Row-by-row loop	● High	✗ Poor
BULK COLLECT only	● Medium	● Moderate
FORALL only	● Medium	● Moderate
BULK + FORALL	✓ Minimal	✓ Best

Tips to Reduce Context Switches

- Use BULK COLLECT with SELECT when fetching multiple rows.
- Use FORALL for batch DML (INSERT, UPDATE, DELETE).
- Use LIMIT in large collections to balance memory and performance.
- Avoid SQL inside PL/SQL loops wherever possible.

Table Functions

A table function is a special kind of PL/SQL function that returns a collection (like a nested table or varray), which can be queried like a regular table using `SELECT * FROM TABLE(function_name(...))`.

This is extremely useful when:

- You need to return multiple rows from a PL/SQL function.
- You want to stream data from PL/SQL logic into SQL queries.
- You're building a pipelined ETL process or dynamic result set generator

Basic Architecture

- Define a collection type (nested table or varray)
- Write a function that returns that collection
- Query it using `TABLE()` in SQL

Table Functions

A *table function* is a function that can be called from within the **FROM** clause of a query, as if it were a relational table. Table functions return collections (nested tables or VARRAYs), which can then be transformed with the **TABLE** operator into a structure that can be queried using the SQL language.

Used for:

- Perform very complex transformations of data, requiring the use of PL/SQL, but need to access that data from within an SQL statement.
- Pass complex result sets back to the host (that is, non-PLSQL) environment.

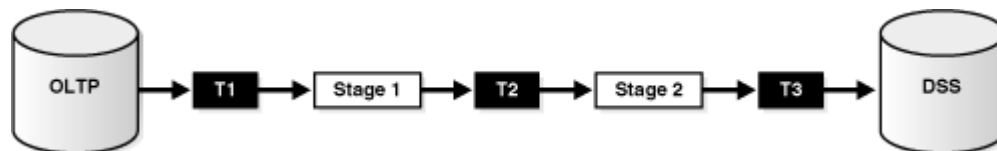
Types:

- *Streaming table functions*

Data streaming means that you can pass from one process or stage to another without having to rely on intermediate structures. Table functions, in conjunction with the **CURSOR** expression, enable you to stream data through multiple transformations, all within a single SQL statement.

- *Pipelined table functions*

These functions return a result set in *pipelined* fashion, meaning that data is returned while the function is still executing. Add the **PARALLEL_ENABLE** to a pipelined function's header, and you have a function that will execute in parallel within a parallel query.



QUALITY. PRODUCTIVITY. INNOVATION.

Create the Table Function

```
CREATE OR REPLACE FUNCTION get_employees_by_dept2 (p_dept_id NUMBER)
RETURN emp_table_type
IS
    v_result emp_table_type := emp_table_type();
BEGIN
    SELECT emp_row_type(employee_id, first_name || ' ' || last_name)
    BULK COLLECT INTO v_result
    FROM employees
    WHERE department_id = p_dept_id;

    RETURN v_result;
END;
/
```

--3. Query the Function Like a Table

```
SELECT * FROM TABLE(get_employees_by_dept2(30));
```

Summary & Homework



THEORY

[PL/SQL 101 Part 9](#)

[PL/SQL Tutorial](#)

PRACTICE

1. [bulk collect](#)
2. [Characteristics of BULK COLLECT Collections](#)

HOMEWORK

1. [Characteristics of BULK COLLECT Collections](#)
2. [Analytic Functions: Databases for Developers](#)

Takeaway

1. [Quiz on Bulk-Fetching from a Cursor Into One Collection of Records](#)